TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 10: Network Flow I: Maximum Flow and Minimum Cut

Foreign Trade University

Contents

1 Introduction: Flow as a Modelling Tool

This week we study one of the most versatile algorithmic frameworks in the entire course: **network flow**. The setting is intuitive – imagine a network of pipes (a directed graph), each pipe with a fixed maximum throughput (a *capacity*), and ask: how much "stuff" (water, traffic, data, abstract units) can we route from a designated *source* to a designated *sink* without exceeding any pipe's capacity? This is the **Maximum Flow** problem.

What makes flow so important is not the pipe story but the astonishing range of problems that turn out to be flow in disguise. Bipartite matching, edge-disjoint paths, project selection, image segmentation, baseball elimination, and many scheduling problems all reduce to a single max-flow computation. Two of this week's exercises (Problems 9 and 11) develop the matching and edge-disjoint-paths reductions directly; next week is devoted almost entirely to such applications.

Underlying everything is a single, beautiful duality result – the **Max-Flow Min-Cut Theorem** – which says that the maximum amount you can push through a network exactly equals the capacity of the cheapest "bottleneck" that separates source from sink. This theorem ties together an *optimization* problem (maximize flow) and a *combinatorial* problem (find the minimum cut), and the proof itself *is* the correctness proof of our main algorithm. We follow Kleinberg & Tardos, Chapter 7.

2 Flow Networks, Flows, and Their Value

2.1 Flow networks

Definition 2.1 (Flow network)**.** *A* flow network *is a directed graph $G = (V, E)$ together with:*

- *a non-negative* capacity $c(e) \geq 0$ *on each edge $e \in E$ (we assume integer capacities unless stated otherwise);*
- *a distinguished* source *node $s \in V$ and* sink *node $t \in V$, with $s \neq t$.*

For simplicity we assume no edge enters s and no edge leaves t, and that there are no parallel edges in the same direction (a pair $u \to v$ and $v \to u$ is allowed).

We think of $c(e)$ as the maximum rate at which material can travel along edge e.

2.2 Flows

Definition 2.2 (s-t flow)**.** *An s-t* flow *is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying two conditions:*

(i) ***Capacity constraints.*** *For every edge e, $0 \leq f(e) \leq c(e)$.*

(ii) ***Conservation constraints.*** *For every node $v \neq s, t$,*

$$\sum_{e \text{ into } v} f(e) \;=\; \sum_{e \text{ out of } v} f(e),$$

i.e. flow in equals flow out (nothing is created or destroyed at internal nodes).

Definition 2.3 (Value of a flow)**.** *The* value *of a flow f, written $v(f)$, is the net flow out of the source:*

$$v(f) \;=\; \sum_{e \text{ out of } s} f(e) \;-\; \sum_{e \text{ into } s} f(e).$$

(With our assumption that no edge enters s, this is just the total flow leaving s.)

The **Maximum-Flow Problem** is: given a flow network, find a flow of maximum value. Problem 1 in this week's exercises asks you to write a *flow feasibility checker* that confirms the two conditions above and returns $v(f)$ – a good way to internalize the definitions before computing flows.

2.3 A first conservation identity

A flow's value can be measured at the source *or* at the sink – they always agree.

Lemma 2.4. *For any s-t flow f, the net flow out of s equals the net flow into t.*

Proof. Sum the conservation equation "flow in = flow out" over *all* nodes $v \neq s, t$, and note that every edge $e = (u, w)$ with $u, w \notin \{s, t\}$ contributes $+f(e)$ once (as flow out of u) and $-f(e)$ once (as flow into w), cancelling. After cancellation, only terms touching s and t survive, and rearranging gives net-out(s) = net-in(t). This is a special case of the more general cut identity in Lemma 3.2. □

3 Cuts and Cut Capacity

Definition 3.1 (s-t cut)**.** *An s-t* cut *is a partition (S, T) of the node set V into two sets with $s \in S$ and $t \in T$ (so $T = V \setminus S$). The* capacity *of the cut is the total capacity of edges directed from S to T:*

$$c(S, T) \;=\; \sum_{\substack{e=(u,v) \\ u \in S,\, v \in T}} c(e).$$

Crucially, only edges going *from S to T* count; edges from T back to S are free. Problem 2 asks you to compute $c(S, T)$ given S.

Cuts give an immediate *upper bound* on any flow: no matter how cleverly you route flow, every unit must cross from S to T at some point, and the $S \to T$ edges can carry at most $c(S, T)$ in total.

Lemma 3.2 (Flow across a cut)**.** *Let f be any s-t flow and (S, T) any s-t cut. Then*

$$v(f) \;=\; \sum_{e \text{ out of } S} f(e) \;-\; \sum_{e \text{ into } S} f(e),$$

where "out of S" means edges from S to T and "into S" means edges from T to S.

Proof. By definition $v(f) = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ into } s} f(e)$. For every node $v \in S$ with $v \neq s$, conservation gives $\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ into } v} f(e) = 0$. Add all these (one per $v \in S$) to $v(f)$; the value is unchanged because we added zeros. Now group the sum by edges. An edge with both endpoints in S contributes $+f(e)$ (out of its tail) and $-f(e)$ (into its head), cancelling. An edge from S to T contributes only $+f(e)$; an edge from T to S contributes only $-f(e)$. The result is exactly the claimed expression. □

Corollary 3.3 (Weak duality)**.** *For any flow f and any s-t cut (S, T), $v(f) \leq c(S, T)$. Consequently,*

$$\max_{f} v(f) \;\leq\; \min_{(S,T)} c(S, T).$$

Proof. By Lemma 3.2, $v(f) = (\text{flow } S \to T) - (\text{flow } T \to S) \leq (\text{flow } S \to T) \leq \sum_{e: S \to T} c(e) = c(S, T)$, using $f(e) \geq 0$ for the first inequality and $f(e) \leq c(e)$ for the second. Taking the max over flows on the left and the min over cuts on the right gives the second statement. □

The Max-Flow Min-Cut Theorem (Section 5) will show this inequality is actually an *equality*.

4 The Ford-Fulkerson Method

4.1 Why greedy alone fails

A first instinct: repeatedly find a path from s to t with spare capacity and push as much flow as the path's bottleneck allows. This is on the right track, but pure greedy gets *stuck*: an early path choice can "block" a better routing, and – with no way to *undo* flow – the algorithm halts below the optimum. The classic example is the diamond network $s \to a$, $s \to b$, $a \to b$, $a \to t$, $b \to t$ (all capacity 1): greedily routing $s \to a \to b \to t$ saturates the middle and leaves value 1, but the true maximum is 2. The fix is the *residual graph*, which lets us cancel previously routed flow. (Problems 5 and 6 both compute the correct value 2 on this instance.)

4.2 The residual graph

Definition 4.1 (Residual graph)**.** *Given a flow f in G, the* residual graph G_f *has the same nodes and the following edges:*

- *For each edge $e = (u, v)$ of G with $f(e) < c(e)$, a* forward *residual edge (u, v) with* residual capacity $c(e) - f(e)$ *– the spare capacity we can still push.*
- *For each edge $e = (u, v)$ of G with $f(e) > 0$, a* backward *residual edge (v, u) with residual capacity $f(e)$ – representing the option to* cancel *up to $f(e)$ units already sent along e.*

In our implementation (see `starter_code.py`), every edge is stored together with a paired reverse edge of capacity 0; pushing d units on an edge does `flow += d` on it and `flow -= d` on its reverse, so residual capacities in both directions stay consistent automatically. Problem 3 asks you to construct G_f explicitly from a flow f.

4.3 Augmenting paths

Definition 4.2 (Augmenting path)**.** *An* augmenting path *is a simple path from s to t in the residual graph G_f. Its* bottleneck *is the minimum residual capacity of its edges.* Augmenting *along it means pushing `bottleneck` units of flow on each of its edges (increasing f on forward edges, decreasing f on backward ones).*

Lemma 4.3. *If f is a flow and P is an augmenting path with bottleneck $b > 0$, then augmenting f along P yields a new flow f' with $v(f') = v(f) + b$.*

Proof sketch. On each forward residual edge we add $b \le c(e) - f(e)$, keeping $f' \le c$; on each backward residual edge we subtract $b \le f(e)$, keeping $f' \ge 0$. Conservation is preserved at every internal node of P because the path enters and leaves the node by exactly one residual edge each, changing in- and out-flow by the same b. The first edge of P leaves s (a forward edge, since no edge enters s), so v increases by exactly b. □

4.4 The Ford-Fulkerson method

Ford-Fulkerson is a *method*, not a fully specified algorithm, because it does not say *which* augmenting path to pick. Problem 6 implements the version that finds augmenting paths by *depth-first search*; Section 6 below pins down the path choice to get a good running-time bound.

Algorithm 1 Ford-Fulkerson (generic augmenting-path method)

```
Initialize f(e) ← 0 for all edges e
while there exists an augmenting path P in the residual graph G_f do
    b ← bottleneck residual capacity along P
    Augment f along P by b (update forward and backward residual edges)
end while
return f
```

5 The Max-Flow Min-Cut Theorem

We now prove the central result. The argument is elegant: when Ford-Fulkerson stops, the nodes reachable from s in the final residual graph define a cut whose capacity equals the flow value – which, by weak duality, forces both to be optimal simultaneously.

Theorem 5.1 (Max-Flow Min-Cut Theorem)**.** *In every flow network, the maximum value of an s-t flow equals the minimum capacity of an s-t cut. Moreover, a flow f is maximum if and only if its residual graph G_f contains* no *augmenting path (no s-t path).*

Proof. We prove the following three statements are equivalent for a flow f:

(i) there is a cut (S, T) with $c(S, T) = v(f)$;

(ii) f is a maximum flow;

(iii) G_f contains no augmenting path.

(i) $\Rightarrow$ (ii). If $c(S, T) = v(f)$ for some cut, then by weak duality (Corollary 3.3) every flow f' satisfies $v(f') \le c(S, T) = v(f)$, so f is maximum (and (S, T) is simultaneously a minimum cut).

(ii) $\Rightarrow$ (iii). Contrapositive: if G_f has an augmenting path, then by Lemma 4.3 we can increase $v(f)$, so f is not maximum.

(iii) $\Rightarrow$ (i). Suppose G_f has no s-t path. Let

$$S \;=\; \{\, v \in V : v \text{ is reachable from } s \text{ in } G_f \,\}, \qquad T = V \setminus S.$$

Then $s \in S$, and $t \notin S$ (else there would be an augmenting path), so (S, T) is a genuine s-t cut. Consider any edge $e = (u, v)$ of the *original* graph with $u \in S$ and $v \in T$. We must have $f(e) = c(e)$ (the edge is *saturated*); otherwise G_f would contain a forward residual edge (u, v), making v reachable from s – contradicting $v \in T$. Likewise, any original edge $e' = (v, u)$ with $v \in T$, $u \in S$ must have $f(e') = 0$; otherwise G_f would contain a backward residual edge (u, v), again making v reachable. Now apply Lemma 3.2:

$$v(f) \;=\; \underbrace{\sum_{e:\, S \to T} f(e)}_{= \sum c(e) = c(S,T)} \;-\; \underbrace{\sum_{e:\, T \to S} f(e)}_{=0} \;=\; c(S, T).$$

This proves (i). Since (i), (ii), (iii) are equivalent, the maximum flow value equals $c(S, T)$ for this particular cut, and by weak duality no cut can be smaller – so it is the *minimum* cut, and max-flow = min-cut. □

The construction in (iii) $\Rightarrow$ (i) is *algorithmic*: after computing a maximum flow, do one BFS from s in the final residual graph; the reached set S is one side of a minimum cut. Problem 8 implements exactly this extraction and verifies $c(S, T) = v(f)$ on random networks.

6 Edmonds-Karp and the Integrality Theorem

6.1 Termination and integrality

Theorem 6.1 (Integrality Theorem)**.** *If all capacities are integers, then Ford-Fulkerson terminates, and the maximum flow it produces is* integral*: $f(e) \in \mathbb{Z}_{\geq 0}$ on every edge.*

Proof. With integer capacities, the all-zero starting flow is integral. By induction, if f is integral, then every residual capacity is an integer, so each augmenting path's bottleneck b is a positive integer, and augmenting keeps f integral. Each augmentation increases $v(f)$ by $b \geq 1$, and $v(f)$ is bounded above by $c(\{s\}, V \setminus \{s\})$ (the capacity of the source cut), so there can be only finitely many augmentations. Hence the method terminates, with an integral maximum flow. □

Integrality is not just a curiosity – it is precisely what makes the matching and edge-disjoint-paths reductions (Problems 9 and 11) work: a unit-capacity flow comes back as a 0/1 assignment that we can read off as "edge chosen" or "edge not chosen." Problem 12 confirms integrality empirically: on integer-capacity networks, the computed flow is an integer on every edge.

6.2 The Edmonds-Karp BFS rule

Generic Ford-Fulkerson can be slow (even non-terminating with irrational capacities, and exponential with adversarial integer choices). **Edmonds and Karp** fix the path-selection rule: *always augment along a shortest augmenting path*, i.e. one with the fewest edges, found by BFS.

Algorithm 2 Edmonds-Karp (BFS augmenting paths)

```
f(e) ← 0 for all e
while BFS from s in G_f reaches t do
    Let P be the BFS shortest s-t path; b ← its bottleneck
    Augment f along P by b
end while
return f
```

Problem 4 implements the BFS that returns one such shortest augmenting path (or `None` when t is unreachable); Problem 5 wraps it into the full Edmonds-Karp algorithm.

Theorem 6.2 (Edmonds-Karp bound)**.** *With the shortest-augmenting-path rule, Ford-Fulkerson performs $O(VE)$ augmentations, each costing $O(E)$ time for the BFS, for a total running time of $O(VE^2)$ – independent of the capacity values.*

Proof idea. Two facts drive the bound. (1) Under BFS augmentation, the BFS distance $\text{dist}_{G_f}(s, v)$ from s to any node v *never decreases* over the course of the algorithm. (2) An edge (u, v) becomes *saturated* (the bottleneck of the chosen path) only when it lies on a shortest path, i.e. $\text{dist}(s, v) = \text{dist}(s, u)+1$; before it can be saturated again, flow must be pushed back along (v, u), which requires $\text{dist}(s, u)$ to have grown by at least 2. Since distances range in $\{0, 1, \ldots, V-1, \infty\}$, each edge can be saturated at most $O(V)$ times, giving $O(VE)$ total saturations, hence $O(VE)$ augmentations (each augmentation saturates at least one edge). Each BFS is $O(V + E) = O(E)$, so the total is $O(VE^2)$. The full distance-monotonicity argument is in Kleinberg & Tardos, Section 7.3. □

7 Putting the Pieces Together: Verification

A theme of this course is *verifying* that an algorithm's output is actually correct, not just trusting it. Network flow is especially well-suited to this:

- **Cross-checking algorithms.** Since the max-flow value is *unique* (it equals the min-cut), two different correct methods must agree. Problem 7 runs Edmonds-Karp (BFS) and DFS Ford-Fulkerson on the same random networks and confirms they always return the same value.
- **Certificate of optimality.** A flow is provably maximum if you can exhibit a cut of equal capacity (Theorem 5.1). Problem 8 extracts that cut and checks $c(S,T) = v(f)$ – a self-certifying proof of optimality.
- **Structural invariants.** Problem 1 checks the defining flow conditions; Problem 12 checks integrality. These act as assertions that any future change to the code must preserve.

8 Summary and Looking Ahead

This week introduced the **network flow** framework and its central duality:

- A **flow network** is a directed graph with edge capacities, a source s, and a sink t; a **flow** respects capacities and conserves material at internal nodes; its **value** is the net flow out of s.
- An s-t **cut** (S,T) has capacity equal to the total capacity of $S \to T$ edges, and *every* cut upper-bounds *every* flow (weak duality).
- **Ford-Fulkerson** repeatedly augments along s-t paths in the **residual graph**, whose backward edges allow flow to be cancelled.
- The **Max-Flow Min-Cut Theorem** states that the maximum flow value equals the minimum cut capacity; the proof simultaneously yields an algorithm for extracting a minimum cut from any maximum flow.
- **Edmonds-Karp** (augment along shortest BFS paths) runs in $O(VE^2)$, and the **Integrality Theorem** guarantees integer flows for integer capacities.

We also saw how to *verify* flows and cuts, and previewed two reductions – bipartite matching (Problem 9) and edge-disjoint paths via Menger's theorem (Problem 11), plus the super-source/super-sink trick for multiple sources and sinks (Problem 10).

Next week (Network Flow II/III, Kleinberg & Tardos Ch. 7 continued) we turn the crank in earnest on **applications of flow**: maximum bipartite matching and Hall's theorem, disjoint paths and Menger's theorem in full, and a gallery of modelling problems (survey design, project selection / image segmentation, baseball elimination) that all collapse to a single max-flow or min-cut computation.